

Fluxo OAuth2 - PKCE/ConnectId

terça-feira, 8 de abril de 2025 21:38

INSTALAÇÃO DO KEYCLOACK

O primeiro passo é instalar o KEYCLOACK como servidor de autenticação na máquina local, use a porta 8081 ao invés da porta default 8080, uma vez que usualmente o Spring Boot usa a porta 8080. Não precisa baixar a imagem, o comando abaixo baixa a imagem e instancia o container.

Usamos também o modo DEV

Por que usar start-dev durante o desenvolvimento?

1. Configuração mínima

- Não exige banco de dados externo (usa banco em memória).
- Permite subir o Keycloak sem configurar previamente o *realm*, *admin user*, etc. (você pode passar tudo por linha de comando ou configurar via admin console depois).

2. Modo voltado para testes

- Habilita opções inseguras (mas úteis no dev), como:
 - Autenticação básica para o admin
 - HTTPS desabilitado (usa HTTP mesmo)
 - Scripts customizados ativados
 - Stack traces visíveis em erros
 - CORS permissivo por padrão

3. Mais rápido para reiniciar e testar

- Ideal para quem está desenvolvendo um app, testando integração com OAuth2, OpenID Connect etc.

```
docker run -p 8081:8080 -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:latest start-dev
```

Parte	Significado
docker run	Executa um container com base em uma imagem.
-p 8081:8080	Mapeia a porta 8080 do container para a porta 8081 da máquina host . Isso significa que você acessará o Keycloak via http://localhost:8081 .
-e KEYCLOAK_ADMIN=admin	Define a variável de ambiente KEYCLOAK_ADMIN como admin, que será o usuário administrador criado na inicialização.
-e KEYCLOAK_ADMIN_PASSWORD=admin	Define a senha do usuário administrador como admin.
quay.io/keycloak/keycloak:latest	Define a imagem usada para o container — neste caso, a versão mais recente do Keycloak disponível no repositório do Quay.io .
start-dev	Inicia o Keycloak em modo de desenvolvimento , o que significa menos validações, configuração simplificada e sem necessidade de banco de dados externo (usa um H2 embutido). Ideal para testes e desenvolvimento local.

```
C:\Users\Acer>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
738d34b0114e   quay.io/keycloak/keycloak:latest    "/opt/keycloak/bin/k..." 4 minutes ago  Up 4 minutes  8443/tcp, 9000/tcp, 0.0.0.0:8081->8080/tcp  optimistic_kalam
```

Para parar o container, via GUI ou "docker stop <container-id>" -> docker stop abc123def456

Para iniciar o container, via GUI ou "docker start <container-id>" -> docker start abc123def456

CONFIGURAÇÃO DO KEYCLOACK

Precisamos criar um realm na aplicação.

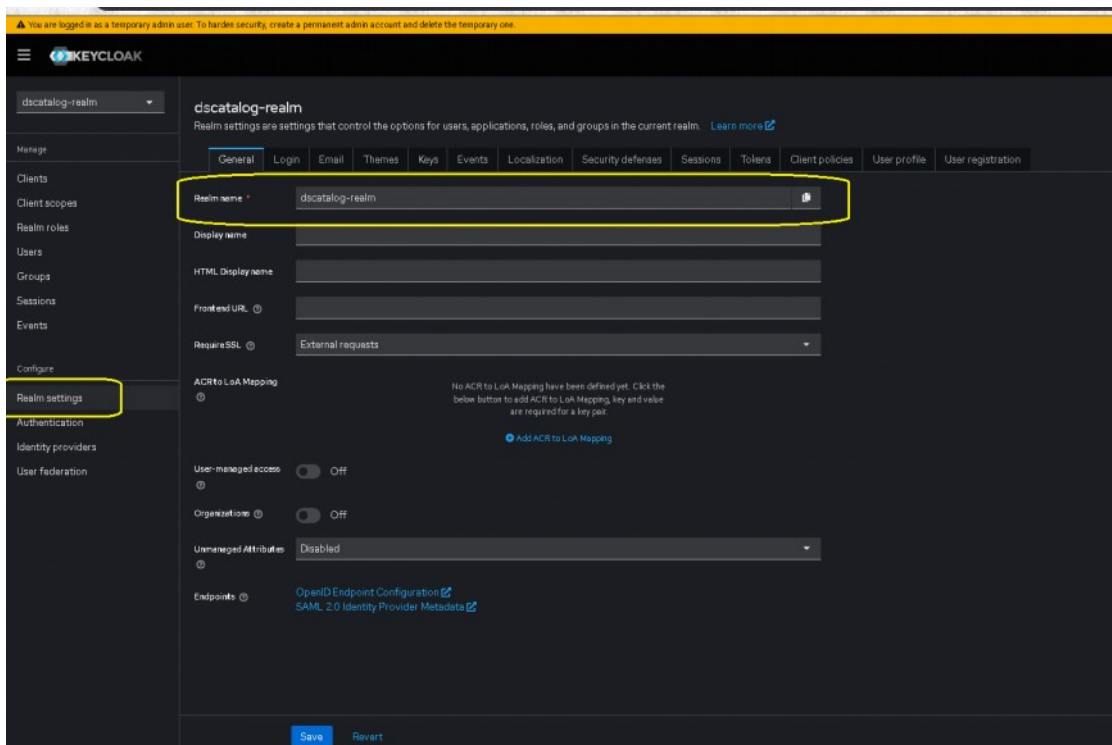
Um Realm é um espaço isolado dentro do Keycloak onde você define:

- Usuários
- Clientes (apps)
- Funções (roles)
- Grupos
- Políticas de autenticação/autorização
- Identidades externas (ex: Google, LDAP, etc)

Ou seja, um realm é como uma "instância" independente de autenticação/autorização.

Criamos um realm "ds-catalog", esse nome será utilizado para os endpoints do Keycloak que serão utilizados pelo Client (no nosso exemplo o Postman).

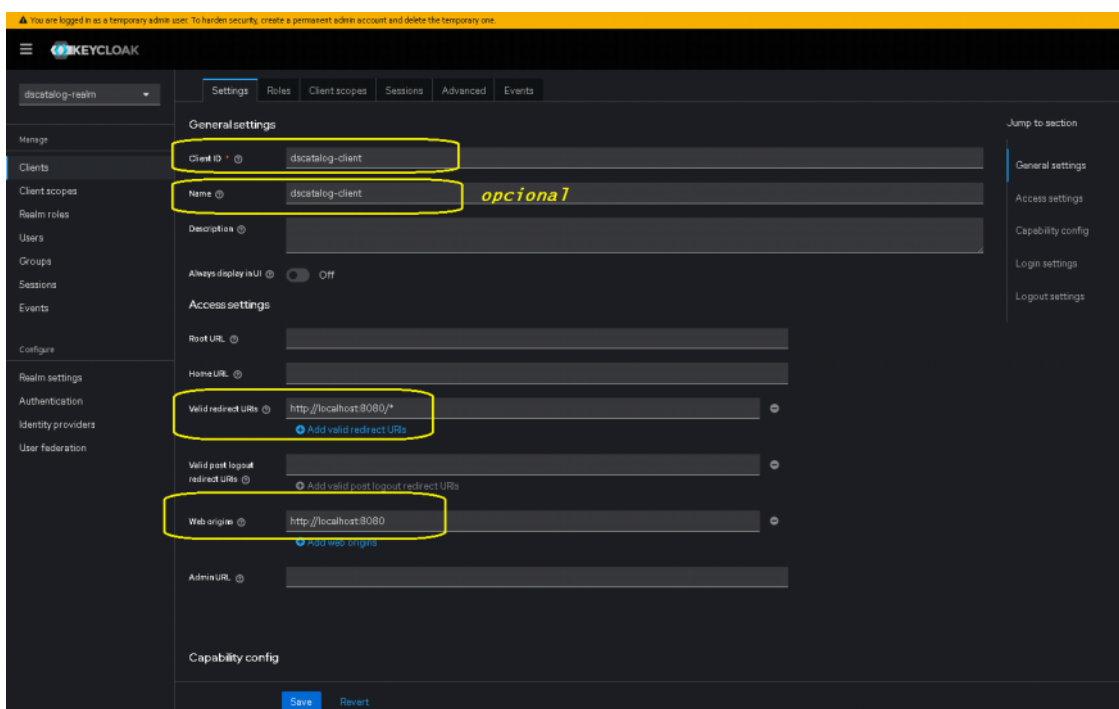
ex: <http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/auth> para autenticação



Criamos então um Client no Keycloak

Um Client é uma aplicação (web, mobile, SPA, API, etc.) que quer usar os serviços de autenticação e/ou autorização do Keycloak. Em outras palavras:

Client = o app que "pede" tokens ao Keycloak para que o usuário possa se autenticar ou acessar recursos protegidos.



Quando você está configurando um SPA (Single Page Application) no Keycloak para usar o Authorization Code Flow com PKCE, você precisa sim configurar exatamente esses três itens no Client (eu configurei "Name" também, mas não é necessário). Vamos detalhar cada um deles:

1. Client ID

- É o **identificador único** da sua aplicação no Keycloak.
- Ex: biblioteca-spa, sistema-academico-frontend, etc.
- Esse ID será usado pela sua aplicação quando fizer requisições ao Authorization Server.

2. Valid Redirect URIs

- São os **endereços para os quais o Keycloak pode redirecionar o usuário** após o login ou logout.
- São obrigatórios no fluxo com PKCE, pois o Authorization Server **nunca deve redirecionar para um URI arbitrário**.
- Exemplos:

<http://localhost:4200/>
<https://app.minhauniversidade.edu.br/>

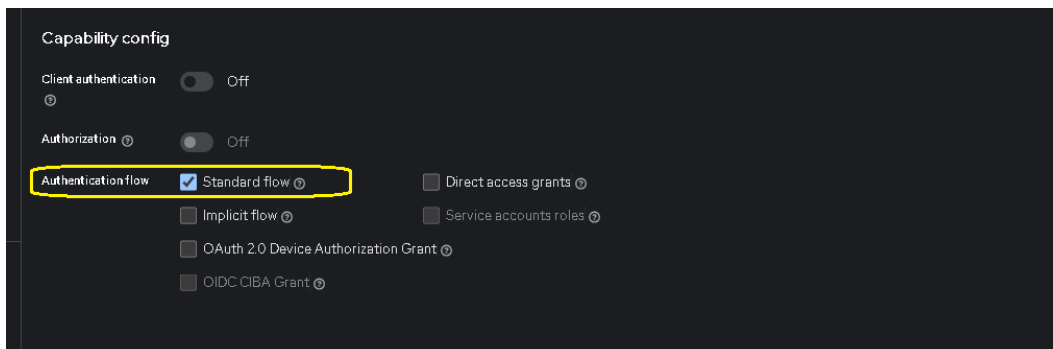
O asterisco no final pode ser usado com cuidado como coringa para subcaminhos.

3. Web Origins

- Define **quais origens (origins CORS)** podem interagir com o Keycloak, importante para SPAs que rodam diretamente no navegador.
- Isso permite que o browser aceite os redirecionamentos e trocas de token feitas entre seu frontend e Keycloak.
- Exemplos:

http://localhost:4200
https://app.minhauniversidade.edu.br

Se deixar esse campo vazio, o Keycloak **bloqueia requisições CORS** da sua SPA.



Client Authentication: OFF

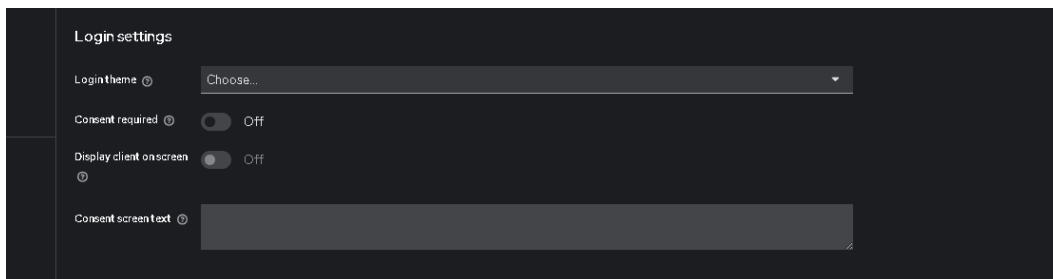
- Como SPAs são **apps públicos (public clients)** que rodam no navegador, **não podem manter segredos com segurança**.
- Por isso, **não exigem autenticação do client** (ou seja, não usam client secret).
- O PKCE substitui o uso do client_secret por um código temporário e seguro.

Authorization: OFF (a não ser que você vá usar o Authorization Services do Keycloak)

- Esse recurso é usado para criar **políticas baseadas em recursos e permissões**, mais comum em apps back-end.
- Para a maioria dos SPAs que só consomem APIs protegidas com scopes e roles, isso pode ficar desativado.

Authentication Flow: Standard Flow (ativado)

- Isso habilita o **Authorization Code Flow**, que é **requerido para uso com PKCE**.
- **Não habilite o Implicit Flow**, pois ele é **menos seguro e não recomendado** atualmente.



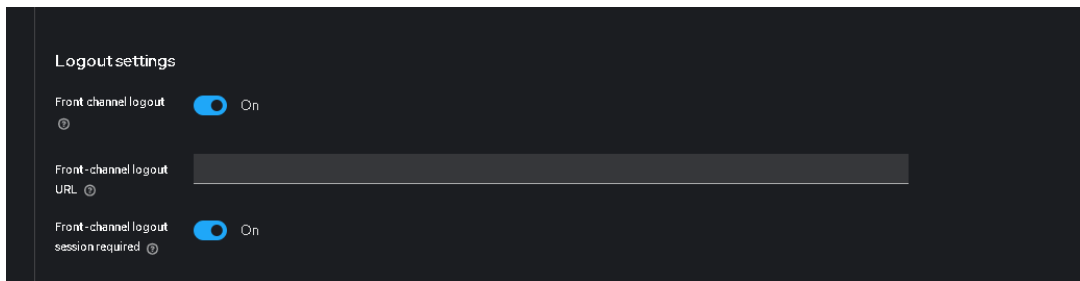
Campo	Valor	Explicação
Login Theme	Choose... (ou default)	Tema visual usado na tela de login do Keycloak. Para desenvolvimento, pode deixar como está.
Consent Required	OFF	Desativa a tela de consentimento do usuário. Boa escolha para SPAs simples.
Display Client Onscreen	OFF	O nome do client não será exibido na tela de login/consent. Isso deixa o visual mais limpo.
Consent Screen Text	(vazio)	Texto adicional mostrado na tela de consentimento, caso ativado. Se Consent Required está OFF, isso é irrelevante.

Consent Required: ON faria sentido quando?

- Em aplicações **de terceiros** que solicitam acesso aos dados do usuário.
- Por exemplo, um app não confiável pedindo acesso à agenda de um usuário no estilo "Este app quer acessar seus dados. Deseja permitir?"
- Em **SPAs de uso interno** (ex: da universidade), isso **não é necessário**.

Quando ativar Display Client Onscreen?

- Quando você **tem múltiplos clients configurados** e quer que o usuário saiba **qual aplicação está solicitando login**.
- Para SPAs de uso claro e único, manter isso desativado ajuda a **simplificar a UX**.



Front Channel Logout: ON

- Isso **habilita o logout "visível" via navegador (front-channel)**, o que é importante para SPAs.
- Quando o usuário faz logout no Keycloak, **o navegador é redirecionado para os clientes registrados** para notificá-los e permitir que façam logout local também.
- Ideal para **sincronizar o logout entre múltiplas aplicações** (ex: se o mesmo login está ativo no sistema acadêmico e no sistema da biblioteca).

Front Channel Logout URL: (deixar vazio ou especificar um endpoint de logout do seu frontend)

- Se deixar **vazio**, o Keycloak vai usar o Valid Redirect URIs como base e chamar a raiz da SPA com parâmetros de logout.
- Se você tiver uma URL específica no seu frontend para tratar o logout (ex: /logout ou /session-end), **você pode especificar aqui**.

Exemplo:

<http://localhost:4200/logout>

⚠ Se não tiver uma rota dedicada no frontend, deixe em branco.

Front-channel Logout Session Required: ON

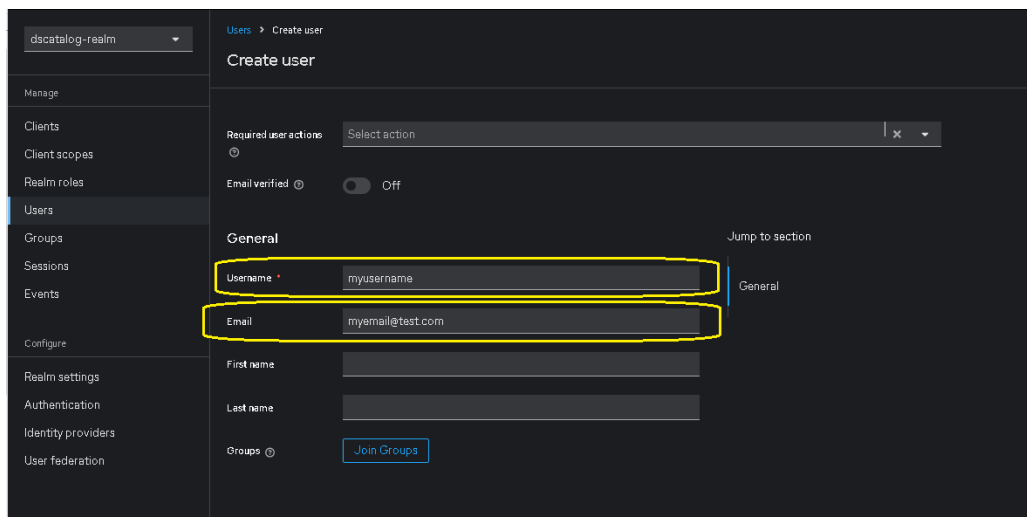
- Garante que o **logout seja feito apenas se a sessão for válida**.
- Isso evita chamadas de logout "falsas" ou feitas por terceiros sem uma sessão real.

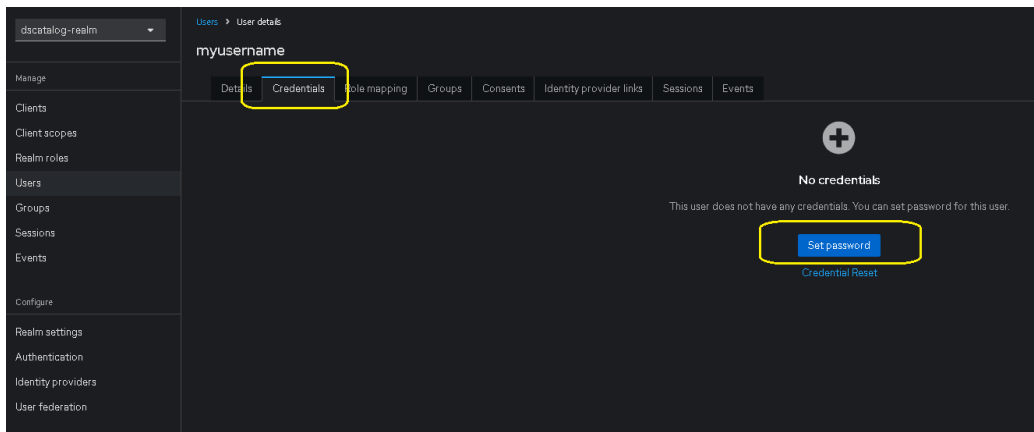
TABS ROLES, CLIENT, SCOPES, SESSIONS e EVENTS -> TUDO DEFAULT

TAB **ADVANCED (IMPORTANTE)** PKCE challenge method : S256, de outra maneira vai ser *plain text*, inseguro.



Criando um usuário (que será solicitado para autenticação)





Users
Users are the users in the current realm. [Learn more](#)

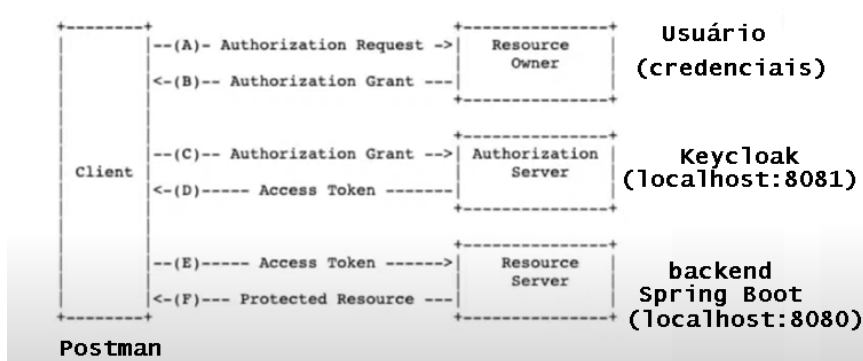
User list

Default search Search user Add user Delete user Refresh

Username	Email	Last name	First name
guest	guest@test.com	Test	Guest
myusername	myemail@test.com	-	-

CONFIGURAÇÃO DO POSTMAN

A arquitetura desse projeto segue a seguinte estrutura:



Authorization Server: Keycloak (localhost:8081), responsável por autenticar o usuário e emitir tokens.

Resource Owner: Usuário, no caso eu, utilizando as credenciais do usuário *guest* com senha 1234.

Resource Server: Aplicação backend Spring Boot

Client: Postman, atuando como um cliente público que solicita autorização e tokens ao Keycloak.

NOTA: NESTA CONFIGURAÇÃO A SEGUIR ESTAMOS VALIDANDO O FLUXO ENTRE O Client e o Authorization Server somente, o Fluxo envolvendo o Resource Server será validado em um momento posterior

1. Configuração Geral no Postman

No Postman, vamos configurar a aba **Authorization** para usar OAuth 2.0 com o fluxo Authorization Code e PKCE. Siga os passos:

- Abra o Postman e crie uma nova requisição (pode ser um GET simples para testar depois).
- Vá até a aba **Authorization**.
- No campo **Type**, selecione **OAuth 2.0**.
- No painel à direita, clique em **Get New Access Token** para configurar os detalhes.

2. Preenchendo os Campos de Configuração

Aqui está como preencher os campos com base nas suas configurações do Keycloak:

- Token Name:** Defina um nome qualquer, como Keycloak Token.
- Grant Type:** Selecione **Authorization Code (With PKCE)**.
 - Justificativa:** Como você desativou Client Authentication (cliente público) e habilitou PKCE com S256, esse é o fluxo correto para garantir segurança em clientes que não mantêm segredo.
- Callback URL:** Insira http://localhost:8080/*.
 - Justificativa:** Este é o Valid Redirect URI que você configurou no Keycloak. O Postman usará isso para receber o código de autorização. Pode deixar como está, mesmo que sua aplicação Spring Boot não esteja no loop agora.
- Auth URL:** Insira <http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/auth>.
 - Justificativa:** Este é o endpoint de autorização do Keycloak para o realm dscatalog-realm. Ele é usado para iniciar o processo de autenticação e obter o código de autorização.
- Access Token URL:** Insira <http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/token>.
 - Justificativa:** Este é o endpoint de token do Keycloak, onde o Postman trocará o código de autorização por um access token.
- Client ID:** Insira o ID do cliente que você criou no Keycloak (exemplo: dscatalog-client). Você pode verificar isso na aba **Clients** do Keycloak Admin Console.
 - Justificativa:** O Client ID identifica o Postman como o cliente que está solicitando acesso.

- **Client Secret:** Deixe em branco.
 - **Justificativa:** Como Client Authentication está desativado, o cliente é público e não requer segredo.
- **Code Challenge Method:** Selecione **SHA-256** (S256).
 - **Justificativa:** Você configurou o PKCE com S256 no Keycloak, então o Postman deve usar o mesmo método para gerar o code_challenge.
- **Code Verifier:** Deixe o Postman gerar automaticamente ou insira um valor manualmente (mínimo de 43 caracteres, como MyCustomCodeVerifierWithAtLeast43Characters12345).
 - **Justificativa:** O code_verifier é enviado ao Keycloak no passo de troca do token, e o servidor verifica se o hash SHA-256 dele corresponde ao code_challenge enviado na solicitação de autorização.
- **Scope:** Insira openid (ou outros escopos que você configurou no cliente, se aplicável).
 - **Justificativa:** O escopo openid é necessário para o protocolo OpenID Connect, que o Keycloak usa por padrão.
- **State:** Pode deixar o Postman gerar automaticamente.
 - **Justificativa:** O state é usado para prevenir ataques CSRF, garantindo que a resposta corresponde à solicitação original.
- **Client Authentication:** Selecione **Send client credentials in body** (embora, neste caso, não haja credenciais para enviar, já que é um cliente público).

Exemplo de configuração no Postman:

Token Name: Keycloak Token
 Grant Type: Authorization Code (With PKCE)
 Callback URL: http://localhost:8080/*
 Auth URL: <http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/auth>
 Access Token URL: <http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/token>
 Client ID: dscatalog-client
 Client Secret: (em branco)
 Code Challenge Method: SHA-256
 Code Verifier: (automático ou manual, ex.: MyCustomCodeVerifierWithAtLeast43Characters12345)
 Scope: openid
 State: (automático)
 Client Authentication: Send client credentials in body

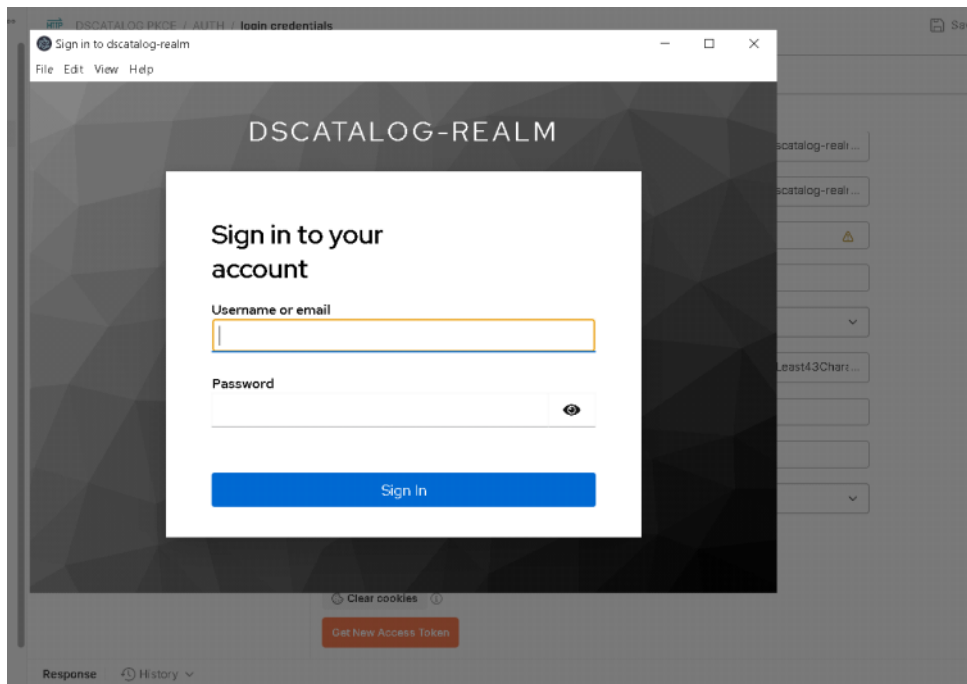
The screenshot shows the Postman interface for a GET request to `{{host}}/oauth/token`. The 'Authorization' tab is active, displaying the following configuration:

- Grant type:** Authorization Code (With PKCE)
- Callback URL:** `http://localhost:8080/*`
- Auth URL:** `http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/auth`
- Access Token URL:** `http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/token`
- Client ID:** `dscatalog-client`
- Client Secret:** (empty)
- Code Challenge Method:** SHA-256
- Code Verifier:** `MyCustomCodeVerifierWithAtLeast43Characters12345`
- Scope:** `openid`
- State:** (empty)
- Client Authentication:** Send client credentials in body

3. Fluxo de Autorização no Postman

1. Authorization Request:

- Clique em **Get New Access Token**.
- O Postman abrirá uma janela no navegador redirecionando para o Auth URL (<http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/auth>).
- O Keycloak exibirá uma tela de login. Insira as credenciais do usuário guest com senha ranger19.
- Após o login, o Keycloak redirecionará para o Callback URL com um código de autorização na query string (ex.: <http://localhost:8080/?code=abc123&state=xyz789>).
- Essa é a primeira etapa do fluxo Authorization Code, onde o Resource Owner (você) autentica-se no Authorization Server (Keycloak) e concede permissão ao Client (Postman).



1. Authorization Grant:

- O Postman captura automaticamente o código de autorização da URL de redirecionamento.
- Em seguida, ele faz uma requisição POST para o Access Token URL (<http://localhost:8081/realms/dscatalog-realm/protocol/openid-connect/token>) com os seguintes parâmetros:

```
grant_type: authorization_code
code: [código recebido]
redirect_uri: http://localhost:8080/
code_verifier: [valor gerado anteriormente]
client_id: dscatalog-client
```

- O código de autorização é trocado por um token de acesso. O PKCE entra em ação aqui: o Keycloak verifica se o hash SHA-256 do code_verifier corresponde ao code_challenge enviado na primeira etapa, garantindo que o pedido é legítimo.

2. Access Token:

- Após a troca bem-sucedida, o Keycloak responde com um JSON contendo o access_token, refresh_token (se configurado), id_token e outros dados, como:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6I..",
  "token_type": "Bearer",
  "expires_in": 300,
  "refresh_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6I..",
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6I..",
  "scope": "openid"
}
```

